Moss 操作系统初赛汇报

基于 StarryOS 的 Linux 兼容性补齐与性能优化

参赛队名：每天睡满八小时

队伍成员：吴勋、张可欣

2026 年 6 月

目录

01 **项目介绍** 项目定位 / 增量范围与目标

02 **核心模块设计** 进程线程 / 调度同步 / 内存 / 文件系统与 I/O / 信号网络 / 平台

03 **验证结果** 兼容性规模、稳定性修复与性能数据

04 **总结与展望** Moss 增量边界、阶段成果与下一步

项目介绍

- moss 是基于 ArceOS 的、以 Starry OS 为基线的 Rust 单体、Linux 兼容操作系统内核项目，目前可以支持 RISC-V 64、LoongArch64 和 AArch64，x86_64 仍在推进
- 2026年3月拉取 https://github.com/Starry-OS/StarryOS 在此基础上进行补充实现，moss 面向真实 Linux 用户态程序的运行需求，持续推进内核功能扩展、关键语义完善与 I/O 性能优化。
- **功能扩展**：syscall 系统调用分发项由基线的 211 项扩展至 264 项；新增文件系统事件通知、异步 I/O、扩展属性、时间服务、调试等能力
- **语义完善**：围绕进程与线程、内存映射、文件描述符、文件系统、信号和定时器，完善 Linux 风格的资源管理与交互语义
- **性能优化**：构建 inode 级共享页缓存，协同普通读写与 mmap；通过预读、批量脏页回写与元数据缓存优化通用 I/O 热路径

Moss 的模块增量总览

方向	StarryOS 基线	Moss 增量
进程线程	fork/clone/exec/waitpid 主路径	clone3、execveat、waitid 与有状态 UID/GID
调度同步	RR、Futex、定时器与等待	覆盖阻塞/唤醒竞争与超时清理
内存管理	地址空间、基础 mmap/COW	mremap、文件映射、SIGBUS 与懒 COW
文件系统	VFS、FileLike 与基本读写	mount/xattr/epoll/事件通知与页缓存
网络模块	基础 TCP/UDP	IPv6 语义与 Loopback 快路径
平台驱动	axcpu/axplat/axhal 抽象	LoongArch64 适配与双架构交付闭环

进程与线程：补齐生命周期与权限语义

StarryOS 基础
fork / clone

→

Moss 增量
execveat + 执行检查

→

退出清理
停止 / 继续状态

→

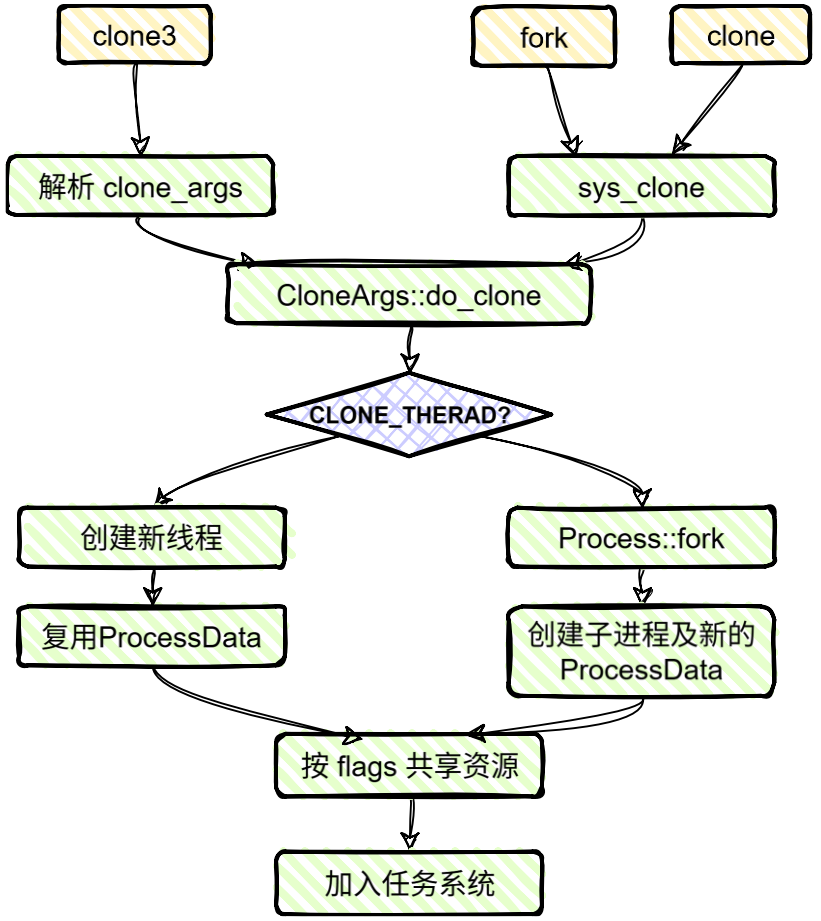
waitid + pidfd
观察与回收

StarryOS 基础

- Thread / ProcessData 双层对象，区分私有状态与共享资源
- 已支持 fork、clone、execve、exit、waitpid 主路径

Moss 重点增量

- clone3 ABI 校验与失败回滚；新增 execveat 与执行约束
- waitid/pidfd 可观察停止、继续与退出
- UID/GID real/effective/saved 已完整实现
- 其余：进程组/会话登记与最小 ptrace 控制链路



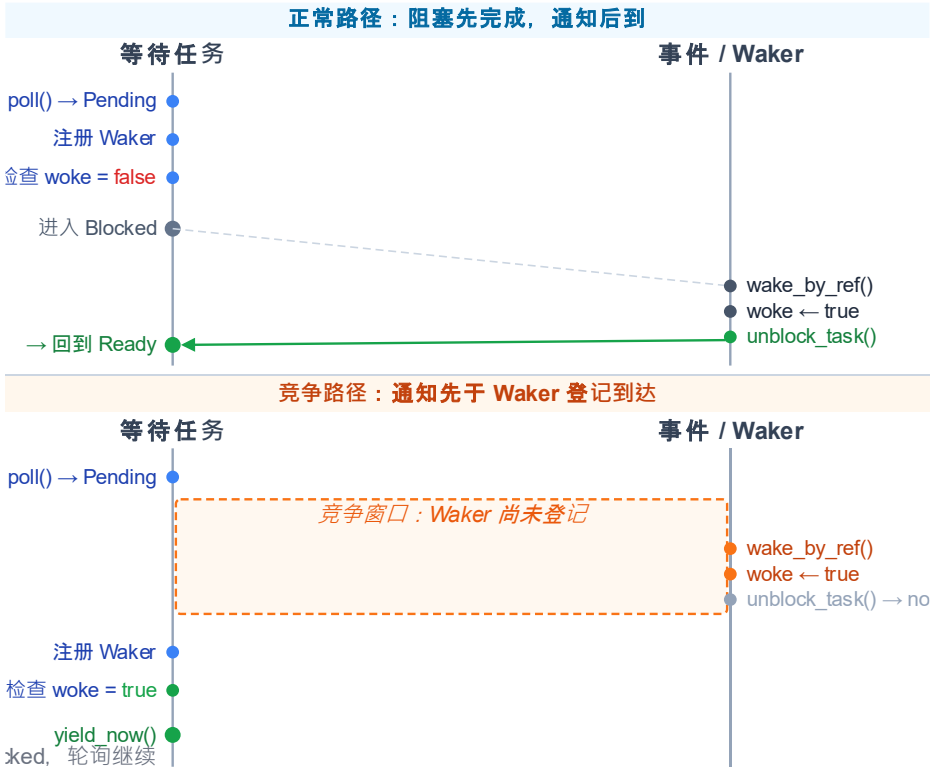
调度与同步：补齐阻塞 / 唤醒语义

StarryOS 基础　已具备 RR、Futex、定时器与等待；Moss 重点处理通知和阻塞交错时的竞争窗口。

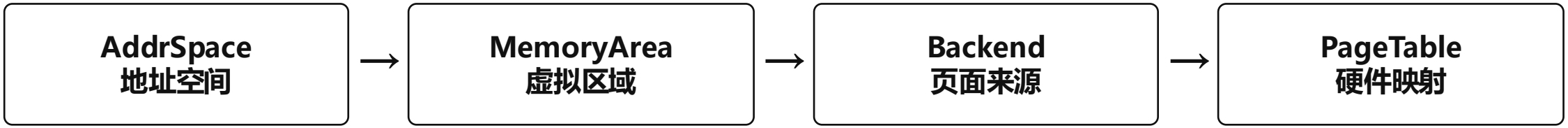
检查条件 → 登记 Waker → 再次确认 → 进入阻塞 → 事件到达 → 恢复复查

Moss 重点增量

- 子进程等待：登记 Waker 后立即复查，可以覆盖退出事件落入窗口的情况
- PollSet：增加 has_waiters 快速判断，取出 Waker 后在锁外 wake
- block_on：woke 标记覆盖“通知先到 / 阻塞先到”
- 定时器：TimerKey = 截止时间 + 序号，重置后旧事件自动过滤；区分相对时长与绝对截止时间
- Futex：独立 notified 状态区分唤醒 / 超时 / 信号中断，精确按地址清理旧等待项



内存管理：补齐映射语义，优化 COW 热路径

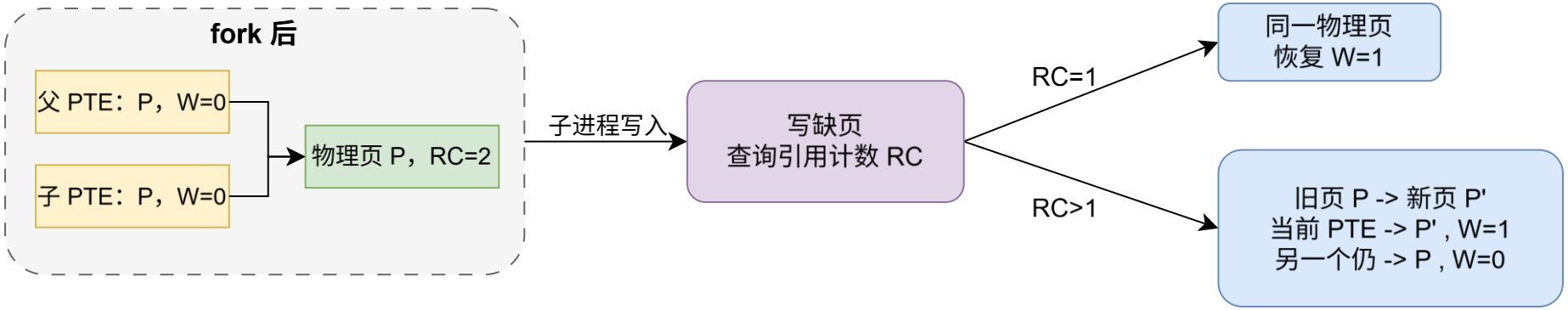


StarryOS 基础

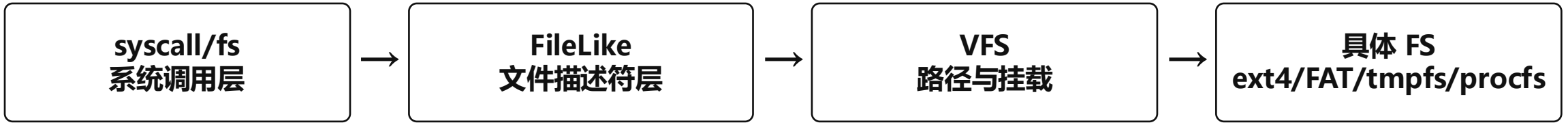
- AddrSpace、MemoryArea、Backend 与 PageTable 已分层
- 基础 mmap、COW 与异常处理可支撑主路径

Moss 重点增量

- 补 mprotect、mremap、文件映射、SIGBUS 等语义
- MAP_GROWSDOWN + 懒分配：按需扩展页面
- COW 路径优化：RC 计数扩展防溢出；未装入区域 fork 时跳过逐页扫描
- 其余：mlock* 与 /proc/[pid]/maps



文件系统与 I/O：从基本 VFS 到语义与性能闭环

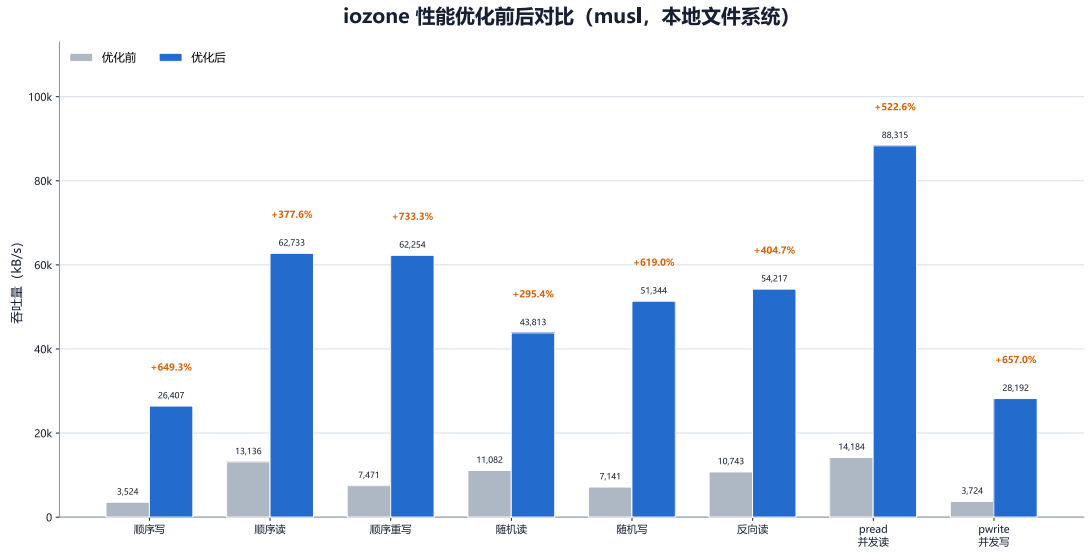


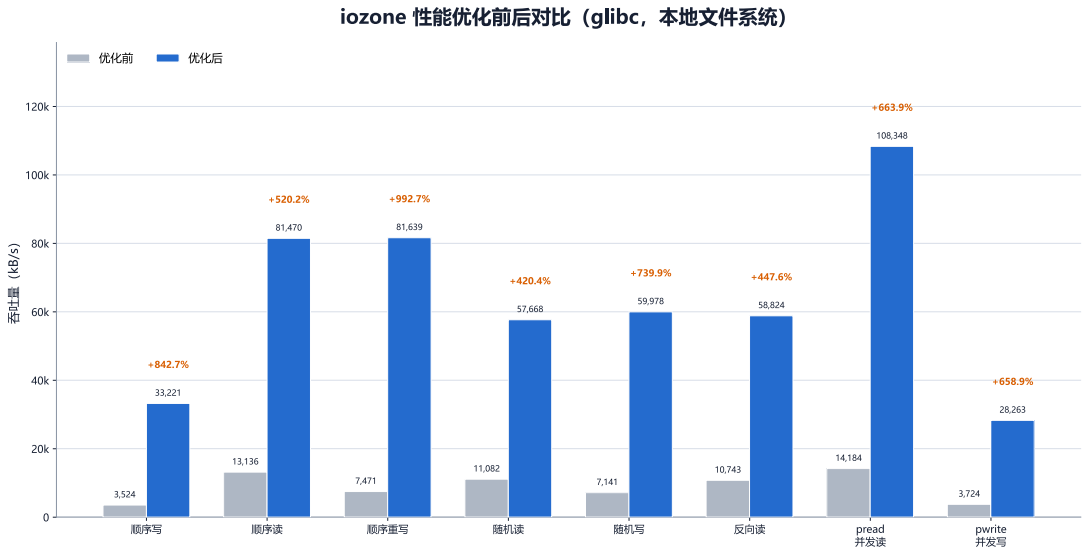
StarryOS 基础

- VFS、FileLike 与基础读写已经可用

Moss 重点增量

- 补齐了新 mount API、xattr、select/poll/epoll
- 重点：inotify/fanotify 覆盖创建、改名、权限等事件
- per-file 页缓存 + 全局索引，统一 read/mmap 缓存
- 元数据缓存、预读、脏页合并写回降低热路径开销
- 其余：O_DIRECT 一致性与 relatime 语义





信号与网络：补关键语义，优化 Loopback 路径

信号：基础与增量

StarryOS 已支持信号投递与屏蔽。
Moss 引入 ProcessWaitStatus / stop_event，使进程具备真正的可暂停状态，waitid可感知停止、继续与退出三类事件；
补充了信号发送的 POSIX 权限校验。

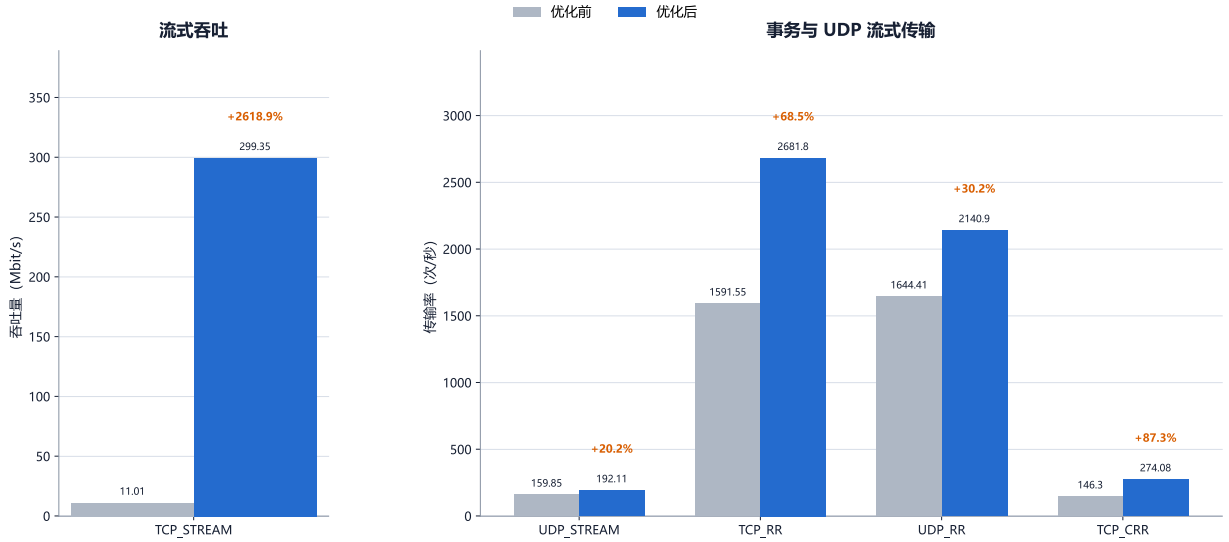
网络：基础与增量

StarryOS 提供了基础的 TCP/UDP。
Moss 显式区分了 IPv4/IPv6，支持 IPv4-mapped IPv6 与 IPV6_V6ONLY；将 TCP/UDP buffer 从 64 KB 分别扩展至 512 KB / 1MB；补齐了 TCP_INFO、SO_SNDBUF/RCVBUF 等 socket 选项。

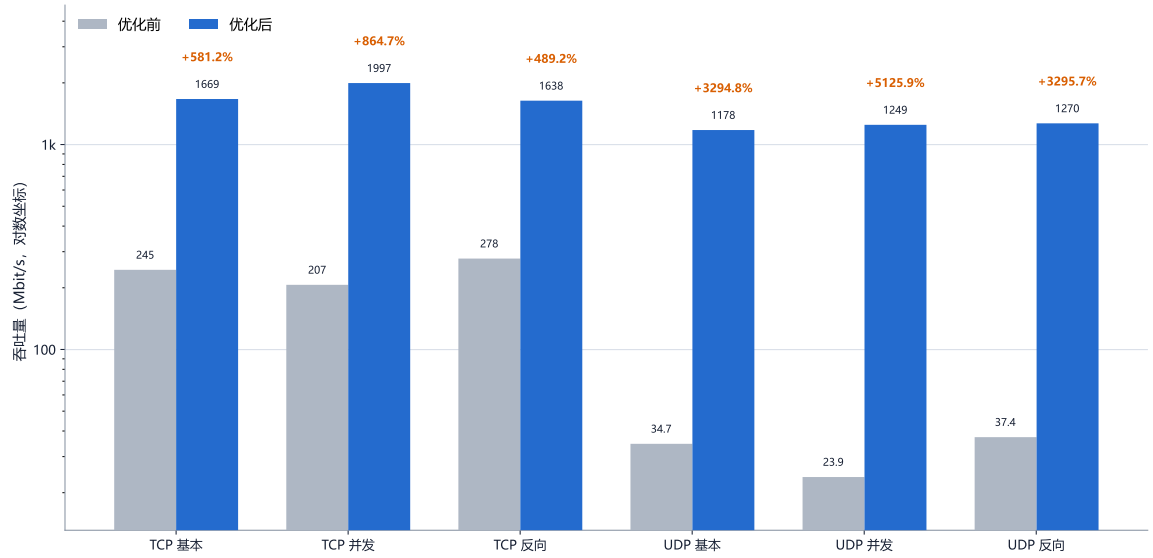
网络重点：Loopback 快路径

Router 直投、MTU 提升至 65535、TCP 绕过拥塞窗口、UDP 直写目标 socket RX queue、按需轮询。RISC-V 回环 TCP 约 0.2–0.28 → 2.1–2.8 Gb/s；UDP 直投后达
1.6–2.27 Gb/s

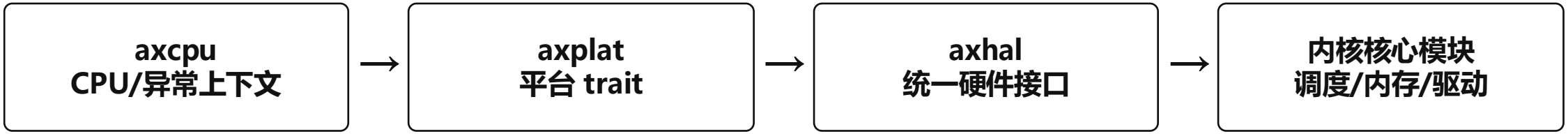
netperf 性能优化前后对比（glibc，本地 Loopback）



iperf3 性能优化前后对比（musl，本地 Loopback）



平台与设备：双架构交付的增量

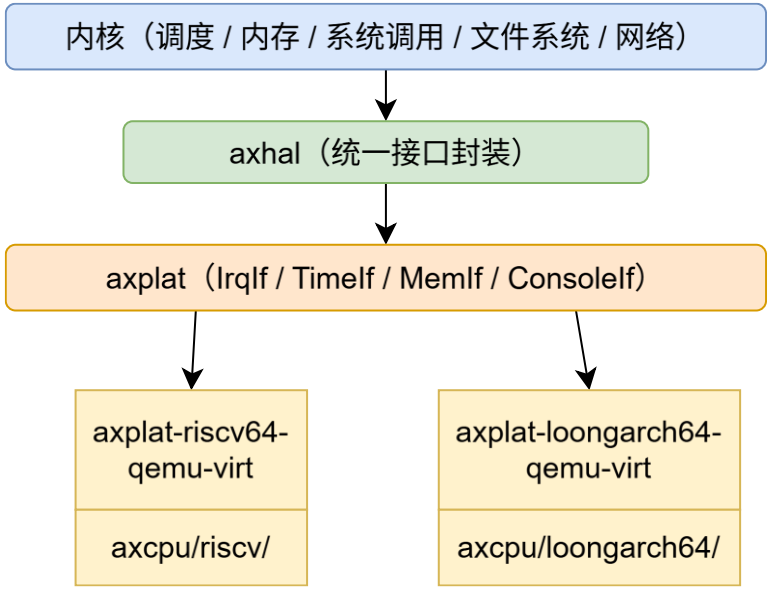


StarryOS 基础

- axcpu / axplat / axhal 已封装 CPU、平台与硬件接口
- 上层沿用统一接口，不直接依赖中断控制器或 CSR 细节

Moss 增量

- 补 LoongArch64 用户态异常与非对齐访问处理
- 生成 RISC-V64 / LoongArch64 内核产物并接入 QEMU 测试
- VirtIO：RV 走 MMIO，LA 走 PCIe，复用 block/net 驱动



增量验证：兼容性、稳定性与性能

兼容性增量

新增 66 个 sys_* 实现：覆盖文件属性与节点管理、挂载与新 Mount API、I/O与事件通知、进程、身份与调度、时间与定时、网络与信号 等。

语义增强

系统性完善 123 个既有 sys_* 实现的代码路径：补齐权限、错误码、阻塞唤醒、资源继承、缓存一致性及进程/内存/I/O/网络边界行为

lmbench

完整代表结果：46.05 → 49.27，提升 3.22（+7.0%）；主要来自 mmap 预读、通知快路径与 select 优化

iozone

glibc 吞吐提升 420%–993%，musl 提升 295%–733%；顺序重写最高提升 992.7%

iperf

TCP Loopback 吞吐提升 472%–865%，由约 200 Mbit/s 提升至 1.6–2.0 Gbit/s
UDP吞吐提升 3106%–6329%，由几十 Mbit/s 提升至约 1.2 Gbit/s

netperf

TCP_STREAM：11.01 → 299.35 Mbit/s，提升 2618.9%，超过 Ubuntu 参考值

以上性能分数均基于本地测试所得，与平台测评机所得分数可能会有不同

总结与展望

阶段性总结

在 StarryOS 基础上，Moss 完成了关键兼容语义、稳定性与性能增量，能够通过一定数量的功能测例和性能测试

后续展望

- 完善 vfork、多线程 exec 收敛与终端作业控制
- 补齐 namespace 隔离与 ptrace 数据访问能力
- 增强脏页写回、msync 与更多文件系统兼容细节
- 优化外部网络流量与多核调度表现
- 固化测试回归与性能对比流程

感谢聆听

敬请批评指正